

オブジェクトを改造する

すでにあるオブジェクトを自分好みに作り変えることを、プログラミングの世界では「拡張（かくちょう / Extension）」や「カスタマイズ」と呼びます。

これは「買った自転車に、ライトをつけたり色をぬったりして改造すること」に例えることができます。

I. オブジェクトを改造する

1. すでにあるオブジェクトを改造する2つの方法

JavaScriptでは、すでに動いているオブジェクトに後から「新しいパーツ」を追加したり、「古いパーツ」を新しいものに取り替えたりできます。

a) 追加する

特定の1人（1つのオブジェクト）だけに、新しいプロパティやメソッドを追加します。

- **イメージ**：自分の自転車だけに、特別なステッカーを貼る。
- **書き方**： `オブジェクト名.新しい名前 = 値;`

```
const myCar = { brand: "Toyota" };

// 後からプロパティを追加
myCar.color = "Red";

// 後からメソッドを追加
myCar.horn = function() {
  console.log("プップー!");
};

myCar.horn(); // 実行できる！
```

[!note]

`myCar.horn = function() { ... }` の部分は、`myCar.horn` に関数を代入しています。
 このように、JavaScriptでは関数も変数のようにあつかうことができます。代入したり、関数の引数にしたり、返り値にしたりできるのです。

b) オーバーライド（上書き）する

親から受け継いだ機能を、自分流のやり方に書き換えます。

- **イメージ**：お父^{とう}さんからもらったカメラだけど、レンズ^{れんず}を最新^{さいしん}のものに交換^{こうかん}して使いやすくする。
- **書き方**：同じ名前^かで新しく定義^{かた}し直す^{おな なまえ あたら ていぎ なお}。

```
// 1. robotオブジェクトを作成
const robot = {
  name: "ロボー",
  greet() {
    console.log("ハロー、ロボットです。");
  }
};

// 2. 実行
robot.greet(); // "ハロー、ロボットです。"（元のまま）

// 3. robot のメソッドを上書（うわが）き（オーバーライド）する
robot.greet = function() {
  console.log("こんにちは！ 私は特別なロボットです。");
};

// 4. 実行して比較（ひかく）する
robot.greet(); // "こんにちは！ 私は特別な..."（上書（うわが）きされた！）
```

[!note]

`robot.greet = function() {...` の部分は、`robot` の `greet` というメソッド（関数）に別の関数を再代入しています。

2. 例え話：スマホ

JavaScriptのオブジェクトは、一度作ったら終わりではありません。

1. **直接追加**：スマホに『お気に入りのストラップ』をつけるようなもの。

2. **オーバーライド**：決まった『着信音』を、自分の好きな音楽に変えるようなものだよ。

こうやって、元の形を壊さずに自分らしく使いやすくなるのが、JavaScriptの面白いところなんです。

II. プロトタイプとプロトタイプチェーン

JavaScriptの「プロトタイプ（Prototype）」と「プロトタイプチェーン」は、多くの学習者がつまづく難しいところです。

これを皆さんがイメージしやすいように、「家系図」や「秘密のレシピの受け継ぎ」に例えて説明します。

1. プロトタイプ（Prototype）とは？

JavaScriptでは、多くのオブジェクトが「親（プロトタイプ）」を持っています。もし自分の中に必要な道具（メソッドやプロパティ）がなくても、「親の持っている道具を借りて使う」ことができる仕組みです。

2. プロトタイプチェーン（Prototype Chain）とは？

「自分^{じぶん}になれば親^{おや}、親^{おや}になればそのまた親^{おや}…」と、さかのぼって探しに行く「鎖^{さが}」のようなつながりのことです。

身近な例え：秘密^{みちか}のカレーレシピ^{たと}を求めて^{ひみつ}

1. あなた（インスタンス）：カレーを作ろうとします。
2. 自分のキッチン：レシピがありません。
3. お父さんの家（自分のプロトタイプ）：「カレーのレシピある？」と聞きます。「持っていない」と答えが返ってきました。
4. おじいさんの家（自分の親のプロトタイプ）：お父さんも持っていなければ、さらにおじいさんに聞きに行きます。「カレーのレシピある？」
5. 先祖の家：そんな風に、レシピを持っている人を見つけるまで、先祖をたどって聞きに行きます。誰かがレシピを持っていれば、あなたはそれを使ってカレーが作れます。

3. なぜこの仕組みがあるの？

もし100人の学生オブジェクトを作るとき、全員に「自己紹介メソッド」を持たせると、メモリーをたくさん使ってしまいます。

- プロトタイプを使わない：全員が自分の辞書を持っている（重い！）
- プロトタイプを使う：教室に1冊だけ辞書を置き、全員でそれを共有する（軽い！）

4. コードで見る「探しに行く流れ」

```
const animal = {
  eats: true
};

const dog = {
  barks: true
};

// dog の親（プロトタイプ）を animal に設定（せってい）
Object.setPrototypeOf(dog, animal);

console.log(dog.barks); // true (自分の中にある)
console.log(dog.eats);  // true (自分にはないが、親にあるので借りてきた！)
console.log(dog.fly);   // undefined (家系図(かけいず)を最後まで探したけど、どこにもない)
```

5. プロトタイプを使って新しいプロパティやメソッドを追加する

ひとつのオブジェクトだけでなく、そのプロトタイプから生まれた「全員」に、新しい機能を持たせることができます。

- **イメージ**：マンションの住人全員が使える「共有のラウンジ」に、新しいテレビを置く。
- **書き方**：クラス名.prototype.メソッド名 = function() { ... };

```
// 全ての配列（はいれつ）（Array）に「2倍にする」という新しい機能を追加してみる
Array.prototype.double = function() {
  return this.map(n => n * 2);
};
```

```
const numbers = [1, 2, 3];
console.log(numbers.double()); // [2, 4, 6]
```

注意：ここではわかりやすい例としてArrayにメソッドを追加しましたが、ふつう、Array や Object などの、皆が使う標準の道具を勝手に改造はしません。他の人の書いたプログラムに影響する可能性があるからです。

まとめテーブル

ようご 用語	やさしい日本語 にほんご	えいご 英語のイメージ
Prototype	きょうつう どうぐぼこ ひみつ 共通の道具箱 / 秘密のレシピ	Blueprint / Shared assets
Prototype Chain	せんそ くさり 先祖（せんそ）をたどる鎖	Search link / DNA
Inheritance	けいしょう おや きのう つか 継承（親の機能を使えること）	Inheritance

「自分にはないものは、先祖から借りればいい」。この考え方がJavaScriptの根本にあります。

学生へのアドバイス

改造はとても便利だけど、やりすぎると『元の形がどうだったか』が分からなくなって、他の人が困ることがあります。
特にArray.prototypeなどを改造するときは、『みんなの共有スペースに勝手に私物を置かない』というマナーがあることも、一緒に覚えておきましょう！

いかがでしょうか？「後から自由に変わられる」というJavaScriptの柔軟な性格が伝わればバッチリです。